

Signed Event Envelope Specification

Version: 1.0.0 Status: Required for 1.0.0

Purpose

Define a canonical, transport-agnostic signed envelope for WID events so verifiers can validate integrity, authenticity, and freshness.

Envelope Shape

```
{
  "version": "1.0",
  "wid": "20260218T164700.0000Z-edge01-event",
  "sig": "<base64url-signature>",
  "key_id": "edge01-ed25519-v1",
  "alg": "Ed25519",
  "issued_at": "2026-02-18T16:47:00Z",
  "expires_at": "2026-02-18T16:52:00Z",
  "data_hash": "sha256:<hex>",
  "meta": {
    "producer": "edge01",
    "scope": "event"
  }
}
```

Required Fields

- **wid**: canonical WID/HLC-WID string.
- **version**: envelope schema version (1.x compatible family).
- **sig**: signature over canonical payload bytes.
- **key_id**: stable key identifier for resolver lookup.
- **alg**: currently Ed25519.
- **issued_at**: RFC3339 UTC timestamp.
- **expires_at**: RFC3339 UTC timestamp; envelope **MUST** be rejected after this time.
- **data_hash**: content hash bound to signature (**sha256:<hex>**).

Canonical Payload To Sign

UTF-8 bytes of this exact newline-delimited tuple:

wid\nkey_id\nalg\nissued_at\nexpires_at\ndata_hash

meta is informational and not part of the canonical signing payload unless explicitly included in **data_hash** derivation by producer policy.

Verification Rules

1. Parse required fields and validate shape.
2. Resolve public key by `key_id`.
3. Verify `alg` is supported (Ed25519).
4. Verify `expires_at` $\geq$ `now_utc`.
5. Rebuild canonical payload and verify `sig`.
6. If external payload is present, recompute and compare `data_hash`.

Any failure $\Rightarrow$ reject envelope.

Conformance Source

- `spec/conformance/signed_envelope.json`
- validated by `tools/check_signed_envelope_spec.py`
- `spec/conformance/signed_envelope_v1.json`
- `spec/conformance/signed_envelope_v1_1.json`
- validated by `tools/check_envelope_compat.py`